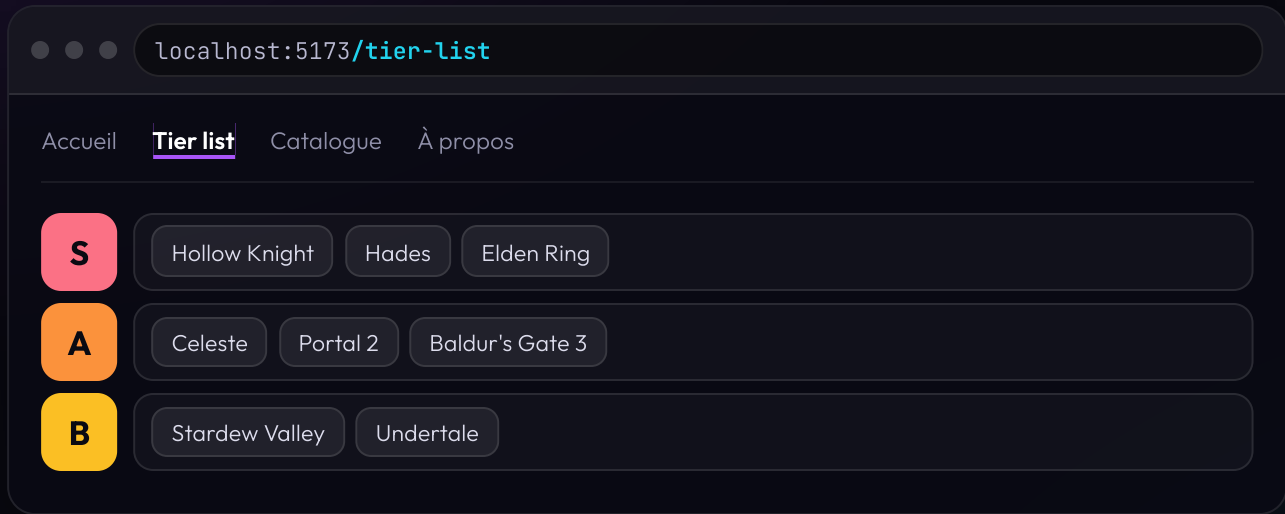


REACT + TYPESCRIPT · 2 HEURES

GameRank

Bonnes pratiques, Tailwind
et navigation entre les pages

Le projet du jour



GameRank — la tier list de vos jeux préférés.

On la construit ensemble, du composant jusqu'aux pages.

Le programme

PARTIE 0 · 15 MIN

ON MONTE LE PROJET

Vite, TypeScript, Tailwind, et une arborescence propre. Vous codez dès la première demi-heure.

PARTIE 1 · 30 MIN

BONNES PRATIQUES

Écrire un composant qu'on peut relire dans trois semaines.

PARTIE 2 · 25 MIN

TAILWIND

Styler sans jamais chercher un nom de classe.

PARTIE 3 · 45 MIN

ROUTAGE

Passer d'une page à cinq, sans recharger.

UN EXERCICE APRÈS CHAQUE PARTIE

On code. Vous repartez avec un projet qui tourne, pas avec des slides. Et un mot de vocabulaire : on écrit du **TSX**, c'est la même syntaxe que le JSX que vous connaissez, dans un fichier TypeScript.



On monte le projet

Avant de parler de code, on ouvre le chantier

Structurer son projet

```
src/  
├─ main.tsx  
├─ App.tsx  
├─ components/  
│   ├─ GameCard.tsx  
│   └─ TierRow.tsx  
├─ pages/  
│   ├─ HomePage.tsx  
│   └─ TierListPage.tsx  
├─ hooks/  
├─ types/  
└─ data/  
    └─ games.ts
```

- `pages/` un fichier par écran. C'est ce que l'utilisateur voit comme « une page ».
- `components/` tout ce qui est réutilisable. Ne sait rien des pages.
- `hooks/` la logique qu'on réutilise à plusieurs endroits.
- `types/` les types partagés entre plusieurs fichiers.
- `data/` les données. Plus tard, l'accès à l'API.

LA RÈGLE QUI EN DÉCOULE

Les pages lisent les données. Les composants les reçoivent en props.

Le setup, en trois étapes

```
npm install tailwindcss @tailwindcss/vite
```

```
// vite.config.ts
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

```
/* src/index.css */
@import "tailwindcss";
```

C'est tout. Pas de fichier de configuration à remplir. Installez l'extension **Tailwind CSS IntelliSense** dans VS Code : elle complète les classes et affiche le CSS correspondant au survol.

EXERCICE 0

Créer le projet

🕒 10 minutes

1. `npm create vite@latest gamerank -- --template react-ts`
2. `npm install`, puis Tailwind avec les trois étapes de la slide précédente
3. Créez les dossiers : `components/`, `pages/`, `hooks/`, `types/`, `data/`
4. `npm run dev`, et mettez une classe Tailwind dans `App.tsx` pour vérifier

CHECKPOINT

Votre page s'affiche sur un fond coloré par Tailwind, et les cinq dossiers existent. À partir de maintenant, tout ce qu'on voit, vous pouvez l'essayer tout de suite.



Bonnes pratiques

Quatre règles, et une arborescence de projet

Pourquoi des règles

SANS

Votre code marche. Vous le relisez dans trois semaines : vous ne comprenez plus rien.

Vous n'osez plus rien modifier de peur de casser.

AVEC

Chaque fichier tient sur un écran. Vous savez où chercher.

Vous ajoutez une fonctionnalité sans relire tout le reste.

Les quatre règles qui suivent ne sont pas des goûts personnels.
Chacune évite un **bug précis** que vous allez rencontrer.

1. Un composant fait une seule chose

séparation des responsabilités

Qu'est-ce qui cloche ?

```
function TierListPage() {  
  return (  
    <>  
    <PageTitre texte="Ma tier list" />  
    <TierRow tier="S" games={jeuxS} />  
    <TierRow tier="A" games={jeuxA} />  
  </>  
)  
}
```

LA RÈGLE

Si vous devez **scroller** pour lire votre composant, découpez-le.

Si vous ne pouvez pas le nommer en un seul mot, il fait deux choses.

2 • Un composant annonce ce qu'il reçoit

typer ses props · jamais `any`

Ce composant affiche du vide. Pourquoi ?

La prop s'appelle `game`, pas `titre` : il fallait écrire `props.game.titre`. Avec `any`, **TypeScript se tait**.
Aucun souligné rouge, juste une page vide.

```
type GameCardProps = {  
  game: Game  
  onSelect?: (slug: string) => void  
}  
  
function GameCard({ game, onSelect }: GameCardProps) {  
  return <h2>{game.titre}</h2>  
}
```

LA RÈGLE

Un type au-dessus de chaque composant. En échange, votre éditeur autocomplète les props et hurle quand vous en oubliez une.

2 • Les props en pratique

déclarer, typer, utiliser

LE COMPOSANT REÇOIT

```
type GameCardProps = {
  titre: string
  note: number
  nouveau?: boolean
}
export default function GameCard(
  { titre, note, nouveau }: GameCardProps
) {
  return (
    <article>
      <h2>{titre} - {note}/10</h2>
      {nouveau && <span>NOUVEAU</span>}
    </article>
  )
}
```

LA PAGE ENVOIE

```
import GameCard from '../components/GameCard'
export default function HomePage() {
  return (
    <div>
      <GameCard titre="Hades" note={9.5} nouveau />
      <GameCard titre="Celeste" note={9} />
    </div>
  )
}
```

TROIS RÉFLEXES

- Le `?` rend la prop **facultative**.
- Guillemets pour une string, accolades pour le reste : `titre="Hades"` mais `note={9.5}`.

3 • Ce qui se calcule ne se stocke pas

le state dérivé

```
function TierListPage({ games }: TierListPageProps) {  
  const [nbJeuxS, setNbJeuxS] = useState(0)  
  const [moyenne, setMoyenne] = useState(0)  
  
  useEffect(() => {  
    setNbJeuxS(games.filter((g) => g.tier === 'S').length)  
    setMoyenne(games.reduce((s, g) => s + g.note, 0) / games.length)  
  }, [games])  
  
  return <p>{nbJeuxS} jeux en S • moyenne {moyenne}</p>  
}
```

Deux problèmes. Lesquels ?

Au **premier rendu**, l'écran affiche `0 jeux en S • moyenne 0`. Et ces deux valeurs peuvent se désynchroniser des données : deux vérités pour la même info.

3 • la version juste

- Pas de `useState`, pas de `useEffect`
- Impossible d'être désynchronisé : il n'y a plus qu'une source
- 4 lignes au lieu de 11

LA RÈGLE

Posez-vous toujours la question : **est-ce que je peux le calculer ?** Si oui, c'est une variable. Pas un state.

COROLLAIRE

Et on ne **modifie jamais** une donnée reçue. `games.sort()` réordonne le tableau d'origine et React ne s'en aperçoit pas. Écrivez `[...games].sort()` ou `games.toSorted()`.

4 · Une liste a besoin d'étiquettes qui ne bougent pas

la prop `key`

Ça marche. Jusqu'au jour où vous **réordonnez** votre tier list.

S

Hollow Knight

Hades

Celeste

Vous montez Celeste en premier. Les étiquettes `0, 1, 2` restent aux mêmes **positions**, mais elles désignent maintenant d'autres jeux. React croit que le jeu n°0 a juste changé de titre : il **garde** son contenu interne. Case cochée, animation, champ de saisie : sur le mauvais jeu.

```
<GameCard key={game.slug} game={game} />
```

LA RÈGLE

La `key` répond à « **est-ce le même élément qu'avant ?** ». Un index répond « est-ce à la même position ? ». Ce n'est pas la même question.

QUIZ · 30 SECONDES

Combien d'erreurs ?

```
function GameCard(props: any) {  
  const [note, setNote] = useState(0)  
  
  useEffect(() => {  
    setNote(props.game.note)  
  }, [props.game])  
  
  return <p>{props.game.titre} - {note}/10</p>  
}
```

Deux. `props: any` au lieu d'un type, et un state qui recopie une prop.

La version juste tient en deux lignes : `{ game }: GameCardProps`, puis directement `{game.note}` dans le TSX.

Pour aller plus loin

Quatre choses à connaître, qu'on ne développe pas aujourd'hui.

- **Sortir la logique du JSX** — un `.filter().sort().slice()` dans le `return` se met dans une variable au-dessus. Le JSX décrit ce qui s'affiche, pas comment on le calcule.
- **La composition avec `children`** — quand un composant accumule six props booléennes, c'est qu'il devrait accepter du contenu à la place.
- **Les union types** — `type Tier = 'S' | 'A' | 'B' | 'C'` plutôt que `string`. TypeScript refuse alors `'Z'` à la compilation.
- **Les hooks personnalisés** — quand la même logique sert dans deux composants, elle sort dans une fonction qui commence par `use`.

Gardez-les dans un coin de la tête. Vous en aurez besoin sur votre projet.

EXERCICE 1

Vos données, vos composants

🕒 10 minutes

1. Dans `types/game.ts`, écrivez le type `Game` : titre, studio, année, note
2. Dans `data/games.ts`, un tableau `games: Game[]` avec **vos** 5 jeux préférés
3. Un composant `GameCard` avec un type de props explicite. Au bon endroit.
4. Affichez la liste dans `App.tsx`, avec une `key` stable

CHECKPOINT

Vos 5 jeux s'affichent. Zéro `any`, zéro `key={index}`, et `GameCard.tsx` est dans `components/`, pas dans `pages/`.

Ne stylez rien pour l'instant : c'est la partie 2.

02

Tailwind CSS

Styler sans chercher de nom de classe

Ce qui fatigue en CSS classique

- **Trouver un nom.** `.card`, `.game-card`, `.card-title`, `.card-title--big` ... vous passez plus de temps à nommer qu'à styler.
- **La cascade.** Votre règle ne s'applique pas, quelque chose l'écrase, vous ajoutez `!important` et vous passez à autre chose.
- **Le CSS mort.** Vous supprimez un composant, sa CSS reste. Six mois plus tard, personne n'ose y toucher.
- **L'aller-retour.** Modifier une couleur = deux fichiers ouverts et un va-et-vient constant.

Tailwind ne remplace pas le CSS. **C'est du CSS**, avec une classe par propriété, et les noms déjà trouvés pour vous.

Le même composant, deux fois

CSS CLASSIQUE · 2 FICHIERS

```
/* GameCard.css */
.game-card {
  display: flex;
  gap: 12px;
  padding: 16px;
  border-radius: 12px;
  background: #1e293b;
}
.game-card__titre {
  font-weight: 600;
  color: #f8fafc;
}
.game-card__studio {
  font-size: 14px;
  color: #94a3b8;
}
```

```
<article className="game-card">
  <h2 className="game-card__titre">{game.titre}</h2>
  <p className="game-card__studio">{game.studio}</p>
</article>
```

TAILWIND · 1 FICHIER

```
<article
  className="flex gap-3 p-4 rounded-xl bg-slate-800"
>
  <h2 className="font-semibold text-slate-50">
    {game.titre}
  </h2>
  <p className="text-sm text-slate-400">
    {game.studio}
  </p>
</article>
```

Zéro nom à inventer. Zéro fichier à ouvrir à côté.
Vous supprimez le composant : le style part avec.

Comment ça marche

Une classe = une propriété CSS. Le nom décrit ce qu'elle fait.

CLASSE	CSS
<code>flex</code>	<code>display: flex</code>
<code>p-4</code>	<code>padding: 1rem</code>
<code>gap-3</code>	<code>gap: 0.75rem</code>
<code>rounded-xl</code>	<code>border-radius: 0.75rem</code>
<code>text-sm</code>	<code>font-size: 0.875rem</code>
<code>bg-slate-800</code>	<code>background: #1e293b</code>

- **Une échelle imposée.** `p-4` vaut 1rem, `p-6` vaut 1.5rem. Vous n'écrivez plus `padding: 13px` par accident : tout l'écran reste cohérent.
- **Des couleurs nommées.** `slate-400`, `slate-800` : une palette de 11 nuances par teinte, déjà équilibrée.
- **Rien d'inutile livré.** Au build, Tailwind lit vos fichiers et ne garde que les classes réellement utilisées.

Vous n'avez pas à retenir les classes. Vous tapez ce que vous voulez faire dans la doc, ou vous laissez l'extension VS Code compléter.

Responsive et dark mode

Un préfixe devant la classe, et c'est réglé.

CSS CLASSIQUE

```
.grille {  
  display: grid;  
  grid-template-columns: 1fr;  
}  
  
@media (min-width: 768px) {  
  .grille {  
    grid-template-columns: repeat(3, 1fr);  
  }  
}  
  
@media (prefers-color-scheme: dark) {  
  .grille { background: #0f172a; }  
}
```

TAILWIND

```
<div className="grid grid-cols-1 md:grid-cols-3  
  dark:bg-slate-900">
```

- ◆ **md:** s'applique **à partir de** 768px
- ◆ **sm:** **md:** **lg:** **xl:** pour les autres seuils
- ◆ **dark:** quand le système est en thème sombre
- ◆ **hover:** **focus:** fonctionnent pareil

Tailwind est **mobile-first** : une classe sans préfixe s'applique partout, et le préfixe ne fait que l'écraser à partir d'une certaine largeur.

EXERCICE 2

Reproduisez cette carte

🕒 12 minutes



Hollow Knight

Team Cherry · 2017

9.5

LE CAHIER DES CHARGES

- Tout en **Tailwind**, aucun fichier `.css`
- Ligne horizontale, éléments centrés verticalement
- Fond sombre, coins arrondis, `padding` confortable
- Le badge : carré, coloré, texte en gras
- Le titre en clair, le studio plus petit et plus terne
- La note poussée à droite
- Au survol : le fond s'éclaircit

Doc ouverte : tailwindcss.com/docs

Pause



10 minutes

03

Routage et navigation

D'une page à cinq, sans jamais recharger

Le problème

Votre application n'a qu'une page. Alors on bricole.

Honnêtement, ça marche. Puis vous testez :

- L'URL reste `localhost:5173`. **Toujours.**
- Le bouton retour du navigateur quitte votre application.
- Vous ne pouvez envoyer **aucun lien** à quelqu'un.
- `F5` et vous revenez à l'accueil.

LE VRAI SUJET

L'URL est un état. Le seul que l'utilisateur peut copier, partager, mettre en favori et recharger. Un router, c'est ce qui la branche sur vos composants.

Deux façons de faire un site

SITE CLASSIQUE

Chaque clic part au serveur, qui renvoie une nouvelle page HTML complète.

Le navigateur jette tout et repart de zéro.
Écran blanc entre chaque page.

APPLICATION MONOPAGE · SPA

Un seul HTML chargé une fois. Ensuite JavaScript réécrit le contenu **et** l'URL.

Aucun rechargement, navigation instantanée.
Votre state React survit.

DÉMO — OUVREZ L'ONGLET RÉSEAU DE VOS DEVTOOLS

Je clique sur un `<a href="/tier-list">`. Regardez la liste des requêtes.

Puis sur un `<Link to="/tier-list">`. Regardez encore.

Les trois briques

Installation : `npm install react-router-dom`

- `BrowserRouter` branche votre app sur l'historique du navigateur. **Une seule fois**, tout en haut.
- `Routes` regarde l'URL et choisit **une seule** route gagnante.
- `Route` associe un chemin à un composant.

`element={<HomePage />}` **avec** les chevrons. Pas `element={HomePage}`. C'est l'erreur numéro un de la séance.

EXERCICE 3

Trois pages, trois routes

🕒 8 minutes

1. `npm install react-router-dom`
2. Dans `pages/`, créez `HomePage.tsx`, `TierListPage.tsx`, `AboutPage.tsx` — un `h1` dans chacune, rien de plus
3. Dans `App.tsx` : `BrowserRouter`, `Routes`, et trois `Route`
4. Les chemins : `/`, `/tier-list`, `/a-propos`

CHECKPOINT

Vous tapez les trois URL à la main dans la barre d'adresse. Les trois répondent.

Naviguer : trois outils, un seul mauvais

JAMAIS DANS UNE SPA

```
<a href="/tier-list">Tier list</a>
```

Rechargement complet. Vous perdez tout votre state React.

LE CHOIX PAR DÉFAUT

```
<Link to="/tier-list">Tier list</Link>
```

Navigation instantanée. Dans le DOM final c'est un vrai `<a>` : le clic droit « ouvrir dans un nouvel onglet » marche toujours.

POUR UN MENU

```
<NavLink  
  to="/tier-list"  
  className={({ isActive }) =>  
    isActive ? 'text-white' : 'text-slate-400'  
  }  
>  
  Tier list  
</NavLink>
```

`NavLink` **sait** s'il pointe vers la page affichée, et `className` accepte une fonction.

Le menu dupliqué

```
<Routes>
  <Route path="/" element={
    <><Menu /><HomePage /><Footer /></>
  } />
  <Route path="/tier-list" element={
    <><Menu /><TierListPage /><Footer /></>
  } />
  <Route path="/a-propos" element={
    <><Menu /><AboutPage /><Footer /></>
  } />
</Routes>
```

Trois fois le même menu. À la quinzième route, vous ajoutez un lien et vous en oubliez deux.

Et pire : à chaque navigation le `Menu` est **démonté puis reconstruit**.

La solution : un composant qui contient la partie fixe, avec un trou au milieu.

Layout et Outlet

```
<Routes>
  <Route path="/" element={<Layout />}>
    <Route index element={<HomePage />} />
    <Route path="tier-list" element={<TierListPage />} />
    <Route path="a-propos" element={<AboutPage />} />
  </Route>
</Routes>
```

La route parente rend le `Layout`. Les trois enfants s'affichent dans son `Outlet`.

DEUX PIÈGES

`index` = la page affichée quand on est **exactement** sur `/`. Et les chemins enfants **n'ont pas** de slash devant : ils se collent au parent.

Une route pour tous les jeux

```
<Route path="jeux/hollow-knight" element={<GameDetailPage />} />  
<Route path="jeux/hades" element={<GameDetailPage />} />
```

Et pour 200 jeux ? Et quand vous en ajoutez un ?

Les deux points déclarent un **paramètre** : un morceau d'URL qui varie.

```
<Route path="jeux/:slug" element={<GameDetailPage />} />
```

/jeux/hades

slug = "hades"

/jeux/celeste

slug = "celeste"

/jeux/nawak

slug = "nawak"

Une seule route, un seul composant, autant de pages que de jeux.

Lire le paramètre : useParams

```
import { useParams } from 'react-router-dom'
import { games } from '../data/games'

export default function GameDetailPage() {
  const { slug } = useParams()

  const game = games.find((g) => g.slug === slug)

  if (!game) {
    return <p>Ce jeu n'existe pas.</p>
  }

  return <h1>{game.titre}</h1>
}
```

`slug` est de type `string | undefined`, et `find()` renvoie `Game | undefined`. D'où le `if (!game)` juste après : il règle les deux cas d'un coup.

Naviguer depuis le code, et gérer le reste

USENAVIGATE

```
const navigate = useNavigate()

<button onClick={() => navigate(-1)}>
  ← Retour
</button>

// navigate('/tier-list')  aller à une page
// navigate(-1)            page précédente
```

LA RÈGLE

L'utilisateur **clique pour aller** quelque part →

`Link`. Le code décide **après une action** →

`useNavigate`.

LA ROUTE 404

```
<Route path="/" element={<Layout />}>
  <Route index element={<HomePage />} />
  <Route path="tier-list" element={<TierListPage />} />
  <Route path="jeux/:slug" element={<GameDetailPage />} />
  <Route path="*" element={<NotFoundPage />} />
</Route>
```

* attrape tout le reste. Sans elle, une mauvaise URL affiche une page **blanche** et vous cherchez un bug qui n'existe pas.

QUIZ · 30 SECONDES

Page blanche. Pourquoi ?

```
<Routes>
  <Route path="/" element={<Layout />}>
    <Route index element={<HomePage />} />
    <Route path="/tier-list" element={<TierListPage />} />
  </Route>
</Routes>
```

Le **slash** devant `tier-list`. Dans une route imbriquée, le chemin se colle au parent : ça donne `//tier-list`, que personne n'atteindra jamais.

Aucune erreur affichée. Aucun warning. Juste une page vide. Retenez-le, vous le ferez au moins une fois.

Le layout et la page détail

🕒 12 minutes

1. `components/Layout.tsx` : un `<nav>`, un `<Outlet />`, un `<footer>`
2. Vos trois liens en `NavLink`, avec un style Tailwind sur le lien actif
3. Passez vos routes en **routes imbriquées** sous le `Layout`
4. Ajoutez `jeux/:slug` → `GameDetailPage` avec `useParams`
5. Chaque `GameCard` devient un `Link` vers son détail
6. Une route `*` → `NotFoundPage`, et testez `/nawak`

CHECKPOINT

Vous copiez l'URL `/jeux/hollow-knight`, vous la collez dans un nouvel onglet : la bonne page s'affiche. Et le menu ne clignote jamais quand vous naviguez.

Récapitulatif

BONNES PRATIQUES

- Un composant, une seule chose
- Des props typées, jamais `any`
- Calculable $\Rightarrow$ variable, pas state
- `key` stable, jamais l'index
- On ne modifie rien en place
- `pages/` `components/` `hooks/`
`types/` `data/`

TAILWIND

- Une classe = une propriété
- Une échelle imposée, donc cohérente
- `md:` pour le responsive
- `dark:` `hover:` `focus:`
- Le style vit avec le composant

ROUTAGE

- `BrowserRouter` une fois, en haut
- `Routes` choisit **une** route
- `Link`, jamais `<a href>`
- `NavLink` pour le menu
- `Layout` + `Outlet`
- `:slug` + `useParams`
- `useNavigate` après une action
- `path="*" toujours`

L'URL est un état. Et si vous pouvez le calculer, ne le stockez pas.

Le TP

2 heures · vous repartez de votre GameRank

LE MINIMUM

1 · La page détail, pour de vrai

Elle existe déjà. Remplissez-la et soignez-la : studio, année, description, jaquette.

2 · Une deuxième tier list : la difficulté

Un nouveau champ dans le type `Game`, une nouvelle page.

3 · Une troisième page, au choix

`/compare/:a/:b` le duel · `/stats` les chiffres

`/tier/:tier` un écran par rang · `/aleatoire` la surprise

LES RÈGLES DU JEU

- Le style est **libre**. Soyez créatifs.
- Librairies autorisées, sauf les kits de composants tout faits.
- L'IA pour **le contenu**, jamais pour le code.
- Date de sortie et studio doivent être **exacts**. Vérifiez-les.
- Les quatre règles du cours s'appliquent aussi ici : un composant une seule chose, des props typées, une `key` stable, et rien de calculable dans un state.